TECH TALK • PASSKEY

告別密碼地獄：打造極致順暢的 Passkey 登入體驗

從一份合約、一顆印章開始，走到 SimpleWebAuthn 的實際程式

章魚燒 • 2026/10/27



密碼地獄：一個「說得出口」的秘密

密碼是你和網站都知道的東西，所以它會被騙、被重複使用、被偷

被釣魚

假網站長得一模一樣，你自己把密碼打進去

重複使用

一個網站外洩，用同一組密碼的其他網站跟著遭殃

資料庫外洩

網站存著跟密碼有關的資料，被偷走就可能被破解

用「簽 PDF 合約」來想像 passkey

把登入想成：你和一家公司長期合作，每次都要簽一份合約



1 **第一次合作，先把驗證工具交給對方**
同時做一顆專屬印章，上面刻著這家公司的名字
註冊

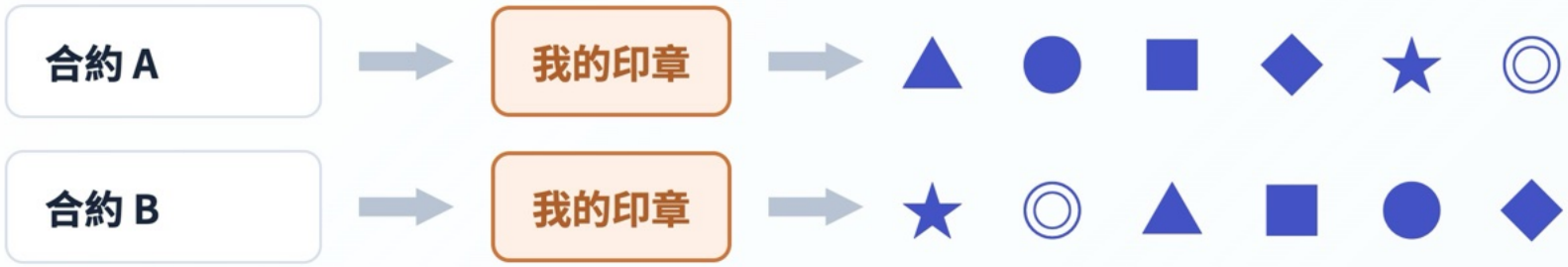
2 **每次要簽，對方寄來一份全新的合約**
內容每次都不一樣，舊的簽名拿來沒有用
challenge

3 **你用只有自己有的印章簽好，寄回去**
印章不離手；寄回時，郵局會在信封蓋上郵戳
私鑰簽章

4 **對方用你當初給的驗證工具確認**
簽章、印章上的公司名、郵戳都對，才算數
公鑰驗證

一顆特別的印章：每次蓋出來都不一樣

要簽的內容不同，蓋出來的排列就不同；只有我的印章蓋得出正確的排列



印章 = 私鑰

只有我有，用來「產生」排列，絕不交給任何人

驗證工具 = 公鑰

發給大家，只能「檢查」排列對不對，造不出新的

數位簽章：私鑰負責簽，公鑰負責驗

驗證是一個數學計算，答案只有「通過」或「不通過」



公鑰可以公開

拿到公鑰也簽不出東西，網站被駭也不怕

改一個字就失效

簽章綁定內容，內容被動過就驗不過

不是「看起來像不像」

不像手寫簽名靠肉眼比對，是數學計算

Live：用 Node.js 親手簽一次

只用 Node.js 內建的 node:crypto，觀察三件事

1

一對鑰匙長什麼樣

公鑰可以直接印出來給大家看，私鑰不行

2

challenge 每次都不同

每執行一次，就產生一份全新的隨機資料

3

簽 A、驗 A → true

換成驗另一份資料，馬上變成 false

```
sign(null, data, privateKey)  →  verify(null, data, publicKey, signature)
```

challenge：每次都是一份全新的合約

合約比喻的第 2 步：網站每次登入都出一道新題目

每次登入，網站都產生一串**全新的隨機資料**，請你當場用印章簽。

- 昨天簽過的那份，今天拿來沒有用（防止重播攻擊）
- 網站出題時先記住題目，收到回覆再比對

EXAMPLE/INDEX.TS

```
req.session.currentChallenge
  = options.challenge;
```

MDN：challenge 是至少 16 bytes 的隨機資料，由伺服器產生

登場角色：一次登入，四個角色

顏色和後面的流程圖一致，看到顏色就知道是誰在做事

使用者 User	瀏覽器 Client	網站後端 RP Server	驗證器 Authenticator
按下按鈕，用指紋、Face ID 或 PIN 確認「是我本人」	傳話人。把「現在在哪個網址」（origin）寫進資料，網頁改不了	出題（challenge）、保存公鑰、驗證簽章。範例裡就是 index.ts	保管私鑰的地方：手機、電腦、密碼管理工具或 USB 安全金鑰

註冊：做一顆新印章，把驗證工具交給網站

先看簡化的四步，接著看實際傳了哪些欄位

1 **網站出題**
產生 challenge，附上 rpID 和使用者資訊
`GET /generate-registration-options`

2 **驗證器做印章**
你按指紋，產生一對新鑰匙，私鑰留在驗證器
`startRegistration()`

3 **交回驗證工具**
送回公鑰，以及含 challenge、origin 的資料
`POST /verify-registration`

4 **網站驗收、存檔**
檢查 challenge、origin、rpID，存下公鑰
`verifyRegistrationResponse()`

註冊第 1 步：網站出的題目長什麼樣？

網站回給瀏覽器的 JSON（GET /generate-registration-options），用範例設定實際產生

欄位	範例的值	意思
challenge	4T05lWW5mz1v…（每次不同）	這次的新合約
rp.id	'localhost'	rpID：印章上刻的公司名稱
user.id／user.name	隨機 ID／'user@localhost'	這把 passkey 屬於誰
pubKeyCredParams	-7（ES256）、-257（RS256）	網站看得懂的簽章演算法，驗證器從中挑一個來做印章
userVerification	'preferred'	要不要確認是本人（UV）
excludeCredentials	[]（第一次註冊是空的）	這個帳號已經有的印章清單，同一個驗證器不會再做第二顆
attestation	'none'	不要求驗證器出示「出廠證明」

註冊第 3 步：交回來的是兩份資料

POST /verify-registration：一份瀏覽器寫，一份驗證器寫

瀏覽器寫的

clientDataJSON

type	'webauthn.create'：這次是註冊
challenge	剛剛那份合約
origin	郵戳

驗證器寫的

authenticatorData

rpIdHash	公司名稱（rpID）的雜湊
flags	UP、UV 等旗標（下一頁）
signCount	蓋章次數（counter）
credential	credential ID + 公鑰

UP 與 UV：蓋章前的兩種確認

記錄在 flags 裡：網站看得到「有沒有確認」，看不到你的指紋

UP User Presence

有人在場

合約比喻：本人親自到場蓋章，不是機器自己蓋

做法：碰一下、按一下、按確認

SimpleWebAuthn 預設一定要有

UV User Verification

確認是本人

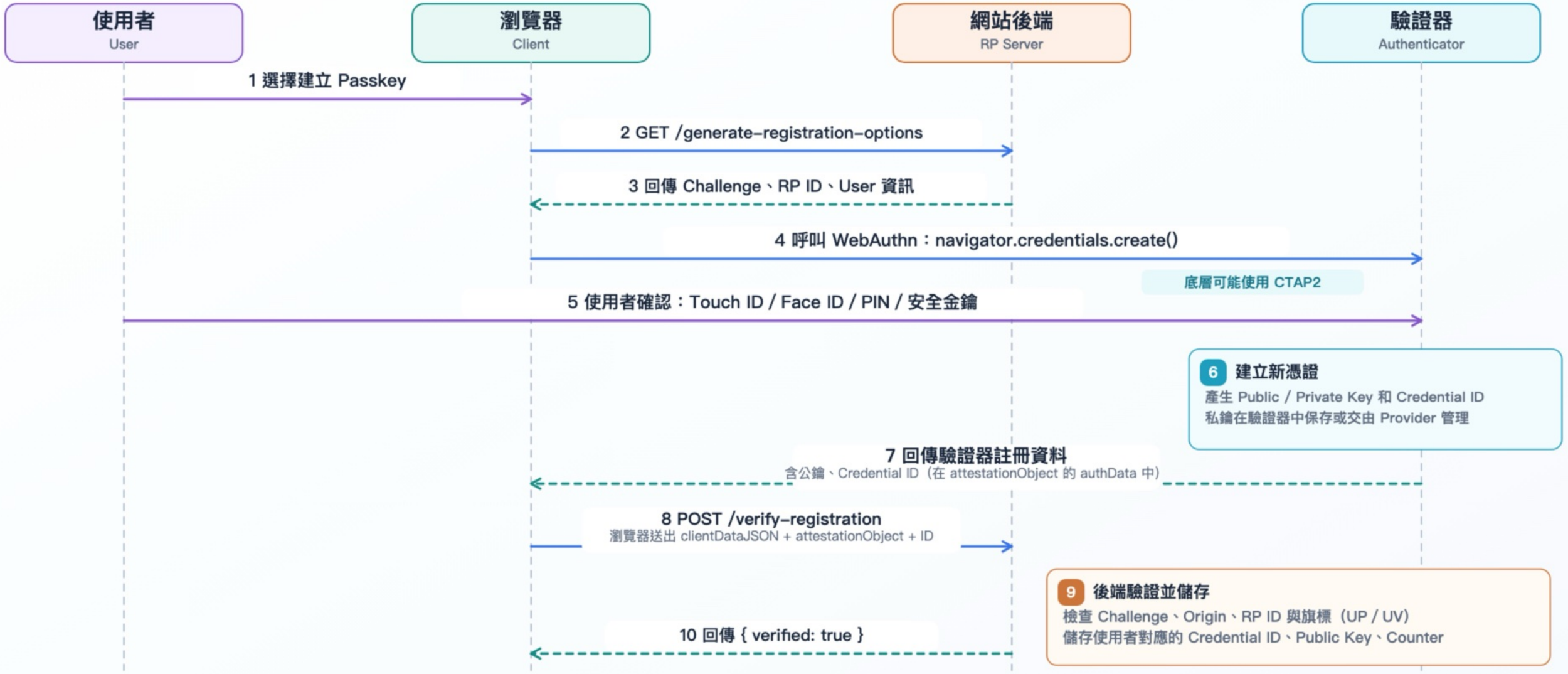
合約比喻：蓋章前，先打開上鎖的印章盒

做法：指紋、Face ID、裝置 PIN 或密碼

網站可設 required／preferred／discouraged

Passkey 註冊流程

從使用者按下「註冊」，到網站儲存 Public Key 與 Credential ID



重點：Private Key 由驗證器 / Passkey Provider 管理，絕不交給網站後端。

登入：蓋章，然後讓網站驗章

不再做新印章，用註冊時存的公鑰來驗證

1 **網站出新題**
新的 challenge，加上這個帳號能用的印章清單
`GET /generate-authentication-options`

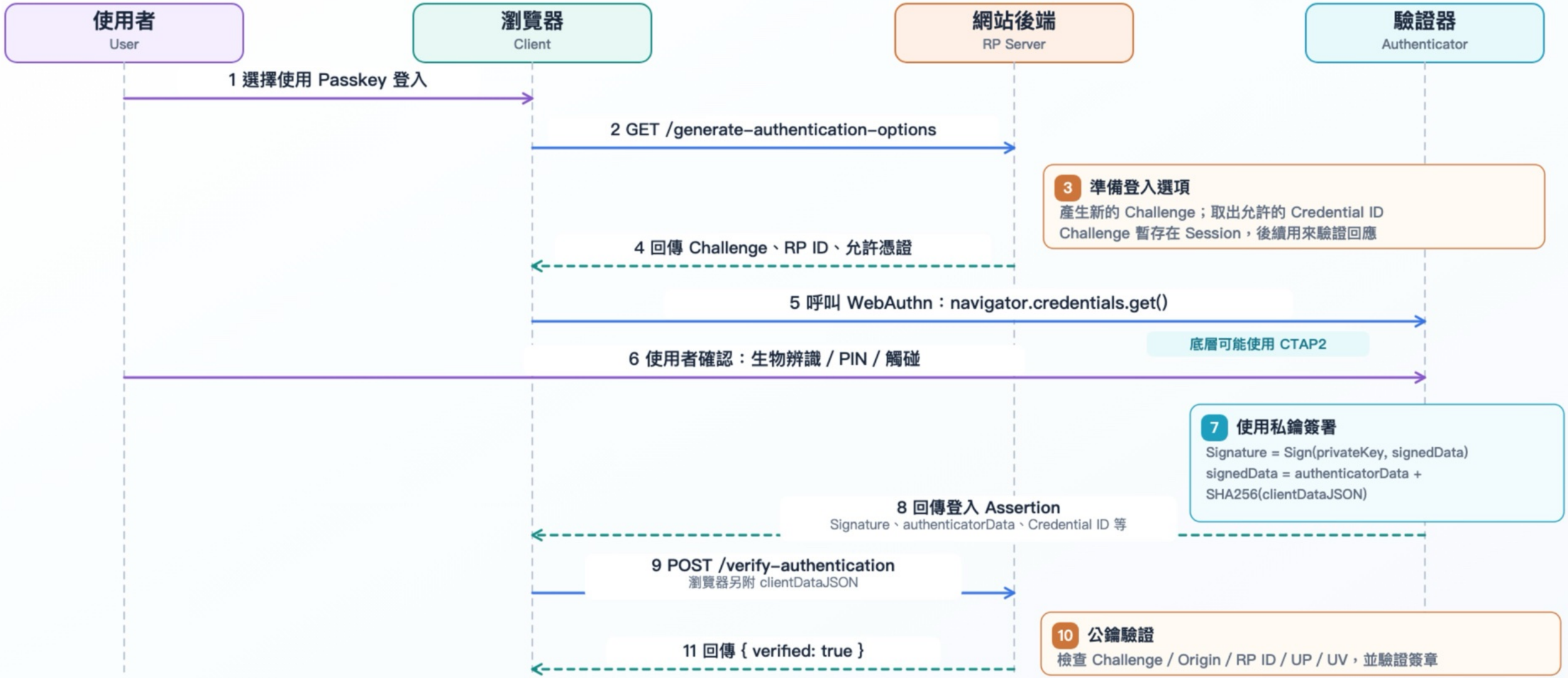
2 **驗證器蓋章**
你按指紋，私鑰對這次的資料簽名
`startAuthentication()`

3 **交回簽章**
簽章，加上被簽的資料本身
`POST /verify-authentication`

4 **網站驗章**
用註冊時存的公鑰驗證，通過就登入
`verifyAuthenticationResponse()`

Passkey 登入流程

登入不再產生新金鑰；後端使用註冊時保存的 Public Key 驗證 Signature



重點：每次登入都需新的 Challenge；WebAuthn 驗證成功 ≠ 自動建立網站登入 Session。

極致順暢：點一下輸入框，passkey 自己跳出來

passkey 自動填入（Conditional UI）



1 **HTML：輸入框標記成 passkey 可用**

```
autocomplete="username webauthn"
```

webauthn 要放在最後一個

2 **JS：頁面一載入就發出登入請求**

```
startAuthentication({
  optionsJSON, useAutofill: true })
```

3 **使用者從選單選 passkey，按指紋就登入**

請求會一直等著，直到使用者選了才繼續

支援：Chrome／Edge 108、Firefox 119、Safari 16 起

MDN：HTML autocomplete 屬性（webauthn） ・ 版本來源：MDN 相容性資料 isConditionalMediationAvailable

關鍵概念

最容易卡住的兩個詞：rpID 與 origin

剛剛流程裡一直出現：印章上刻的公司名稱，和信封上的郵戳。現在把它們講清楚。

origin：這個請求從「哪個網址」發出來

合約比喻：寄回合約時，信封上的郵戳

`http://`	`localhost`	`:8000`
協定 scheme	主機 host	埠號 port

三個部分都一樣，才是同一個 origin

換成 https、換網域或換 port，都是不同的 origin

郵戳是郵局蓋的，不是寄件人寫的

瀏覽器把它寫進要簽的資料，網頁自己改不了

rpID：刻在印章上的「這顆屬於哪個網站」

rpID 只是一個網域名稱：不含 https://，也不含 port

合約比喻：印章上刻著公司名稱，只能簽這家公司的合約。

之後只有在**網域相符**的網站上，瀏覽器才會把這顆印章拿出來。

範例：`rpID = 'localhost'`

假設網頁在 https://login.example.com

rpID 設成	可以嗎？
login.example.com	可以（自己）
example.com	可以（上層網域）
other.com	不行（別人的）
com	不行（公共後綴）

rpID 設 example.com 時，login. 和 shop.example.com 都能用同一顆印章

一張表分清楚 rpID 與 origin

rpID 說「這顆印章是誰的」，origin 說「這次在哪裡蓋的」

	rpID	origin
長什麼樣	example.com	https://login.example.com
誰決定	網站（伺服器設定）	瀏覽器（你實際在哪個網址）
合約比喻	印章上刻的公司名稱	信封上的郵戳
誰來核對	瀏覽器挑印章、網站再核對	網站核對
放在哪裡	authenticatorData 的 rpIdHash	clientDataJSON 的 origin
範例程式	'localhost'	'http://localhost:8000'

點進假網站 examp1e.com，會發生什麼事？

passkey 防釣魚靠的是兩道關卡

第一道關卡 · 瀏覽器

rpID 對不上，印章不會拿出來

假網站沒辦法要求 example.com 的印章，瀏覽器只提供店名相符的 passkey

第二道關卡 · 網站後端

郵戳是假網站的，驗證失敗

就算有簽章被轉送過來，origin 寫的是 examp1e.com，而且已經簽進去改不掉

網站驗章時，逐項檢查這五件事

SimpleWebAuthn 的 verifyAuthenticationResponse() 幫你全部做完

檢查	印章比喻	沒通過代表
challenge 是我剛出的題？	是這次的新合約	有人拿舊簽章來重播
origin 是我的網址？	郵戳正確	在別的網站蓋的
rpIdHash 是我的 rpID？	印章刻的店名正確	是別家網站的印章
counter 沒有倒退？	印章沒被複製	驗證器可能被複製
公鑰驗得過簽章？	驗證工具說 OK	不是本人，或資料被改

範例程式裡，rpID 和 origin 只是兩個設定值

example/index.ts 節錄

```
const { RP_ID = 'localhost' } = process.env;
export const rpID = RP_ID;
expectedOrigin = `http://localhost:${port}`;

await verifyAuthenticationResponse({
  response: body,
  expectedChallenge, // 這次的合約
  expectedOrigin,    // 郵戳
  expectedRPID: rpID, // 店名
  credential: dbCredential, // 公鑰
});
```

常見踩雷

用 127.0.0.1:8000 開網頁，主機不是 localhost，跟 rpID 對不上，瀏覽器直接報錯

正式上線

rpID 改成你的網域，expectedOrigin 改成 https:// 開頭的正式網址

LIVE DEMO

Demo

SimpleWebAuthn 範例專案：註冊、登入、自動填入

常見問題

Passkey 有什麼問題？

聽完之後，大家最常冒出來的五個疑問。

Q1　沒有指紋或 Face ID，就不能用嗎？

可以用。使用者驗證（UV）不只有生物辨識

UV 不等於生物辨識

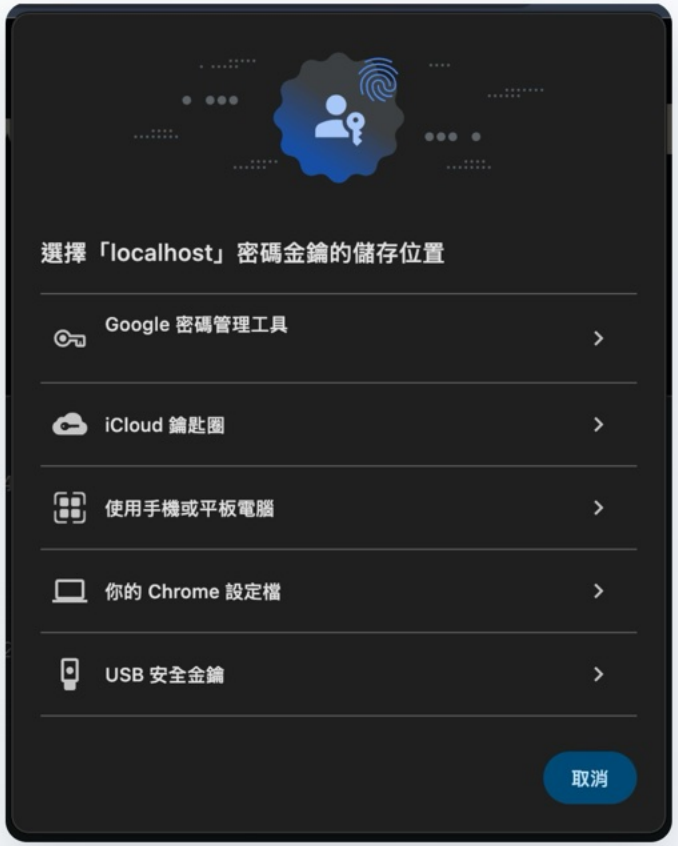
輸入裝置的 PIN 或密碼，也算通過使用者驗證

電腦沒有指紋辨識？

用手機掃 QR code 登入，或插 USB 安全金鑰 —— 右圖就是 Chrome 給的選項

網站可以決定要不要強制

範例設成 userVerification: 'preferred'，有就做、沒有也不擋



Q2　Passkey 只能用在網頁嗎？

不是。App 也能用，還可以跟網站共用同一把 passkey

平台	使用的 API	最低版本
網頁	WebAuthn（navigator.credentials）	主流瀏覽器都支援
iOS／macOS App	AuthenticationServices	iOS 16、macOS 13 起
Android App	Credential Manager	Android 9（API 28）起

iOS：App 和網站互相認證

網站放 apple-app-site-association，App 設定 webcredentials 網域

Android：Digital Asset Links

網站放 /.well-known/assetlinks.json，寫上 App 的套件名稱與簽章指紋

Q3　用不同指紋，可以登入不同帳號嗎？

不行。指紋只負責「解鎖」，不負責「分辨帳號」

指紋在做什麼？

只在裝置上確認「這個人可以解鎖這台裝置」。網站從頭到尾看不到你的指紋。

那帳號怎麼分？

每個帳號各有自己的一把 passkey。登入時從清單選要用哪個帳號。

Q4　私鑰遺失了怎麼辦？

要看 passkey 存在哪裡

同步型 passkey

換新手機，登入帳號就回來了

存在 iCloud 鑰匙圈、Google 密碼管理工具等，會在你的裝置之間同步

綁定裝置型 passkey

丟了就沒了，網站也救不回來

存在 USB 安全金鑰或單一裝置上；網站只有公鑰，沒辦法還原私鑰

網站端要做的事

讓一個帳號可以註冊多把 passkey

保留安全的帳號復原方式

Q5　Passkey 認的是裝置，還是瀏覽器？

都不完全對：認的是「保管它的地方」（passkey provider）

存在哪裡	可以在哪裡用
iCloud 鑰匙圈	同一個 Apple 帳號的裝置；Mac 上的 Safari、Chrome 都能用
Google 密碼管理工具	登入同一個 Google 帳號的 Chrome、Android
Chrome 設定檔	只有這台電腦的這個 Chrome 設定檔，不同步
USB 安全金鑰	插在哪台電腦，就能在哪台用
手機或平板	手機當驗證器，掃 QR code 幫附近的電腦登入

總結：一份合約，四個步驟

和開頭同一個故事，現在每一步都對得上技術名詞

1

第一次合作

做一顆專屬印章，把驗證工具交給對方

私鑰 公鑰 rpID

2

寄來新合約

每次登入，網站都出一道新題目

challenge

3

蓋章寄回

打開印章盒、蓋章，信封上有郵戳

UV 簽章 origin

4

對方核對

簽章、公司名稱、郵戳都對，就登入

公鑰驗證

THANK YOU

謝謝聆聽

歡迎提問 · 章魚燒

延伸閱讀：passkeys.dev · MDN Web Authentication API · simplewebauthn.dev

參考資料

規格、官方文件與範例專案

W3C：Web Authentication Level 3
w3.org/TR/webauthn-3

MDN：AuthenticatorAssertionResponse.signature
developer.mozilla.org › AuthenticatorAssertionResponse

web.dev：Passkey form autofill
web.dev/articles/passkey-form-autofill

Android：Credential Manager 前置作業
developer.android.com › credential-manager

Chrome：Passkeys on iCloud Keychain
developer.chrome.com › passkeys-on-icloud-keychain

Authsignal：Passkey 裝置遺失與復原
authsignal.com › passkey device lost

MDN：Web Authentication API
developer.mozilla.org › Web_Authentication_API

passkeys.dev：What are passkeys／Terms
passkeys.dev/docs/reference/terms

Google：Passkey supported environments
developers.google.com › supported-environments

Apple：Supporting passkeys
developer.apple.com › supporting-passkeys

Corbado：WebAuthn rpID 說明
corbado.com › webauthn-relying-party-id-rpid

SimpleWebAuthn（範例專案）
simplewebauthn.dev ・ github.com/MasterKale/SimpleWebAuthn